AVINASH PAL • LINUX SYSTEMS ENGINEERING

Think Like a Linux Kernel Developer

Mastering C Through Real Linux Kernel Design Patterns

TOPIC: KERNEL PATTERNS & C

TABLE OF CONTENTS (FULL EBOOK SYLLABUS)

1. Why the Linux Kernel Uses C
2. Memory, Pointers, and Kernel Virtual Spaces
3. Structures: Alignment, Padding & Real Kernel Structs
4. Function Pointers: Mechanics & Jump Tables
5. Object-Oriented C and Kernel Subsystem Architecture
6. Putting Everything Together: A Linux Driver Walkthrough
7. Appendix — Linux Kernel Coding Style and Review Checklist

CHAPTER 1 EXCERPT & TECHNICAL NOTES

Author: Avinash Pal | Target: Linux 6.x LTS

1. The container_of() Macro Explained

In Linux kernel C, data structures are intrusive (e.g., struct list_head embedded inside records).
Syntax: container_of(ptr, type, member)
It uses offsetof() to calculate the parent struct pointer from an embedded member pointer safely.
typeof(((type *)0)->member) provides compile-time type checking across GCC and Clang.

2. Driver Lifecycle & Operations Tables

• module_init() / module_exit() register and cleanup device drivers.
• struct file_operations defines callbacks for open(), read(), write(), and unlocked_ioctl().
• Locking rules: use spinlocks for interrupt contexts; use mutexes for sleeping contexts.
• Memory: kmalloc(..., GFP_KERNEL) vs kmalloc(..., GFP_ATOMIC) in bottom half handlers.

3. Production Checklist for Kernel Submissions

• Run scripts/checkpatch.pl --strict on all commits before posting to LKML.
• Always check return values of copy_to_user() and copy_from_user() against EFAULT.
• Never sleep with a spinlock held or inside interrupt handlers.